

1/5

WHAT ACTUALLY HAPPENS WHEN YOU TYPE **google.com**?

The incredible journey from your browser to Google's servers and back!

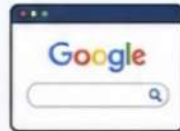


1/5

MOST PEOPLE THINK:



You Type
google.com



Website Opens
Magic! ✨

BUT THE REALITY IS:



A single website visit triggers **multiple** network protocols working together.



QUESTION:

How does your laptop know where Google lives?



FUN FACT

In just a few seconds, your browser will communicate with **multiple** servers across the world!

1 DNS RESOLUTION "Where is Google?"



Your browser asks a DNS server:

What is the IP address of **google.com**?



DNS server responds with the IP address.

```
$ dig google.com
;; ANSWER SECTION:
google.com. 299 IN A 142.250.190.14
```

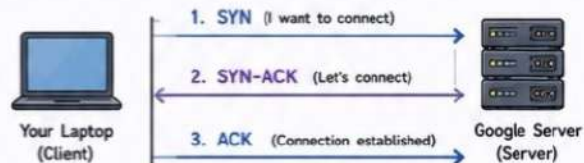


KEY TAKEAWAY: DNS translates domain names (**google.com**) into IP addresses.

2 TCP HANDSHAKE "Can We Talk?"



Before any data is sent, TCP establishes a reliable connection.



KEY TAKEAWAY: TCP handshake ensures both sides are ready to send and receive data.

3 HTTPS & ENCRYPTION "How Is Your Data Protected?"



TLS (SSL) handshake creates a secure, encrypted connection.



- ✓ Browser verifies Google's certificate (issued by trusted CA)
- ✓ Both sides agree on encryption methods
- ✓ A secure encrypted tunnel is established

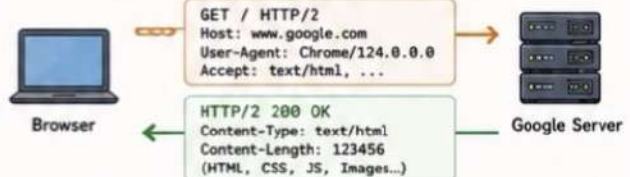


KEY TAKEAWAY: HTTPS encrypts your data so no one can read or tamper with it.

4 HTTP REQUEST & RESPONSE "Let's Get the Data!"



Now the browser sends an HTTP request and gets a response.



KEY TAKEAWAY: Your browser asks for resources and Google's server sends them back.

5 THE COMPLETE JOURNEY From URL to Webpage



FUN FACT: A single page load can trigger: DNS requests, multiple TCP connections, TLS handshakes, and hundreds of HTTP requests for CSS, JS, images, fonts, and more! 🚀



SAVE THIS POST
You'll want to refer back to this later!



QUESTION FOR YOU
Which step surprised you the most?
Comment below! 💡

NEXT UP (Bonus Preview)
How Google Finds the FASTEST Server Closest to You? 🌐



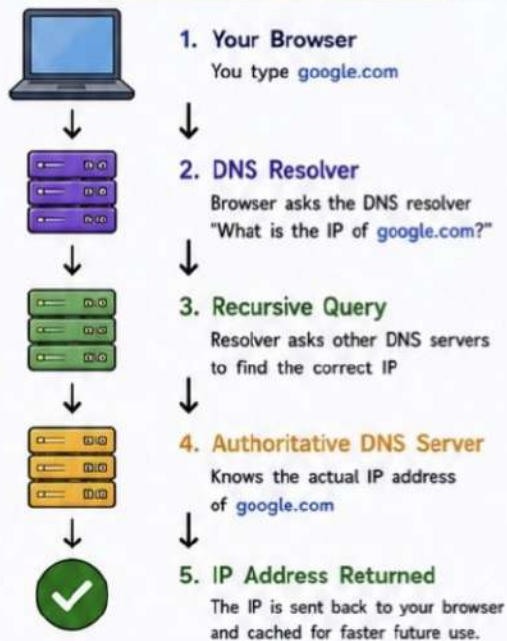
STEP 1: DNS RESOLUTION

"Where is Google?"

Before your browser can connect, it needs an IP address.

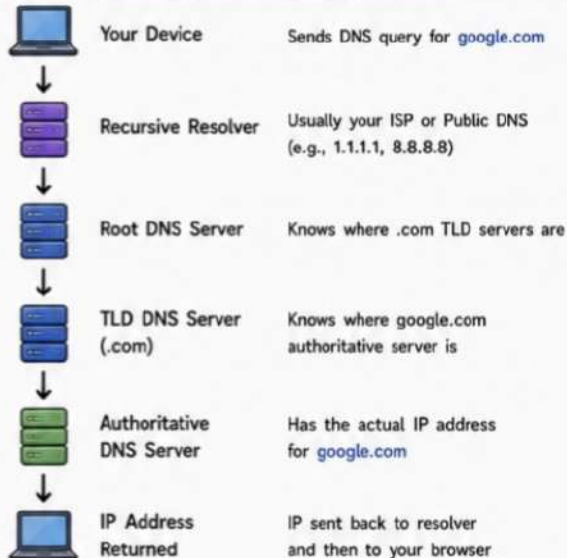


HOW DNS RESOLUTION WORKS



DNS is like the Internet's phonebook. It translates domain names into IP addresses.

DNS HIERARCHY (THE CHAIN)



KEY TAKEAWAY



DNS is a **distributed system**. Many servers work together to find one IP address.

REAL EXAMPLE

```
$ dig google.com
; <<>> DiG 9.18.1 <<>> google.com
;; global options: +cmd
;; QUESTION SECTION:
;google.com.      IN      A
;; ANSWER SECTION:
google.com. 299 IN A 142.250.190.14
;; Query time: 21 msec
;; SERVER: 8.8.8.8#53(8.8.8.8)
;; WHEN: Fri May 17 10:20:30 IST 2024
;; MSG SIZE rcvd: 55
```

Here, 142.250.190.14 is the IP address of `google.com`

COMMON DNS RECORD TYPES

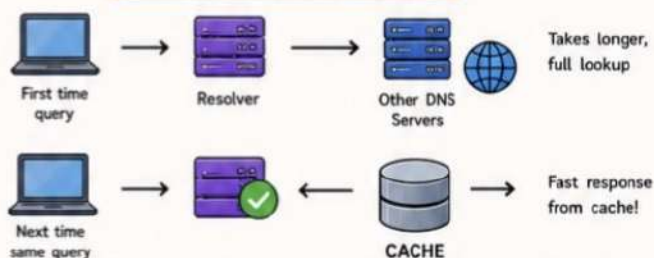
TYPE	MEANS	EXAMPLE
A	Maps domain to IPv4 address	<code>google.com. A 142.250.190.14</code>
AAAA	Maps domain to IPv6 address	<code>google.com. AAAA 2607:f8b0:4005:805::200e</code>
CNAME	Alias of another domain	<code>www.google.com. CNAME google.com.</code>
MX	Mail exchange server	<code>google.com. MX 10 ASPMX.L.GOOGLE.COM.</code>
NS	Name server of the domain	<code>google.com. NS ns1.google.com.</code>
TXT	Text information (SPF, verification)	<code>google.com. TXT "v=spf1 include:_spf.google.com ~all"</code>

USEFUL COMMANDS

<code>dig google.com</code>	Detailed DNS lookup	🔍
<code>nslookup google.com</code>	Simple DNS lookup	🔍
<code>host google.com</code>	Quick DNS lookup	🔍
<code>dig google.com +trace</code>	See the full DNS resolution path	🔍
<code>dig google.com ANY</code>	Get all DNS records	🔍

`dig` is the most powerful DNS troubleshooting tool.

DNS CACHING (SPEED BOOSTER)



Caching reduces load and makes the internet faster!



WHY IT MATTERS?

Without DNS, you'd have to remember IPs instead of names!



QUESTION FOR YOU

Which DNS server do you use most?
1.1.1.1 or 8.8.8.8 or your ISP?
Comment below! 🗨️



NEXT STEP (3/5)

Now that we have the IP, let's establish a connection using TCP Handshake!



STEP 2: TCP HANDSHAKE

"Can We Talk?"

Before any data is sent, TCP establishes a reliable connection.



WHAT IS TCP?

TCP (Transmission Control Protocol) is connection-oriented. It ensures:

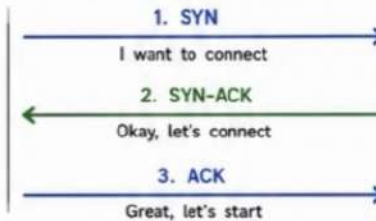
- ✓ Reliable delivery
- ✓ Ordered delivery
- ✓ No data loss (with retransmission)

💡 TCP handshake is like a phone call: Both sides agree to connect before talking.

THE 3-WAY HANDSHAKE



Your Laptop (Client)



Google Server (Server)
IP: 142.250.190.14
Port: 443

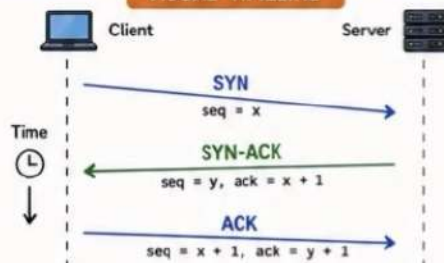


After this 3-step handshake, the connection is **ESTABLISHED** and data can flow in both directions.

DETAILED STEPS

STEP	FROM	TO	FLAG	WHAT HAPPENS?	WHY?
1	Client	Server	SYN	Client sends SYN (Synchronize) packet to the server. <code>seq = x</code>	Client wants to start a connection.
2	Server	Client	SYN-ACK	Server replies with SYN-ACK. It acknowledges the request and sends its own sequence number. <code>seq = y, ack = x + 1</code>	Server is ready to connect.
3	Client	Server	ACK	Client sends ACK to acknowledge the server's response. <code>seq = x + 1, ack = y + 1</code>	Connection is confirmed. Now, data can flow!

VISUAL TIMELINE



💡 This completes the TCP connection establishment. Now the communication channel is open!

REAL EXAMPLE (CAPTURE)

Using: tcpdump or Wireshark

No.	Time	Source	Destination	Info
1	0.000000	192.168.1.10	142.250.190.14	TCP 54321 → 443 [SYN] Seq=1000
2	0.020123	142.250.190.14	192.168.1.10	TCP 443 → 54321 [SYN, ACK] Seq=2000 Ack=1001
3	0.020456	192.168.1.10	142.250.190.14	TCP 54321 → 443 [ACK] Seq=1001 Ack=2001



Source port (54321) is ephemeral (random). Destination port 443 is HTTPS.

WHY 3-WAY HANDSHAKE?

- ✓ Prevents old duplicate connections
- ✓ Synchronizes sequence numbers
- ✓ Ensures both sides are ready
- ✓ Establishes reliability from the start



IMPORTANT NOTE

- ⚠ If any step is lost, the connection is not established.
- ⚠ Firewalls may block these packets.
- ⚠ Use timeouts & retries if no response.

Check open connections:

```
$ ss -tulnp
$ netstat -tnp
```

COMMON TCP PORTS YOU SHOULD KNOW

22	80	443	3306	5432	6379	25	53
SSH	HTTP	HTTPS	MySQL	PostgreSQL	Redis	SMTP	DNS



Port 443 (HTTPS) is used by Google to serve secure web pages.



KEY TAKEAWAY

TCP handshake creates a reliable, ordered and loss-free connection before any data is exchanged. ✓



QUESTION FOR YOU

What happens if the third ACK packet is lost? How will TCP handle it? Comment below! 💡



NEXT STEP (4/5)

Great! Connection is ready. Next, let's see how HTTPS encrypts everything!



STEP 3: HTTPS & ENCRYPTION

"How Is Your Data Protected?"

Now that we have a TCP connection, let's make it secure with TLS.



WHAT IS HTTPS?

HTTPS = HTTP over TLS (Transport Layer Security)
It encrypts the data between your browser and the server so no one can read or modify it.



Encryption
Keeps data private



Integrity
Ensures data is not tampered



Authentication
Proves you are talking to the real server

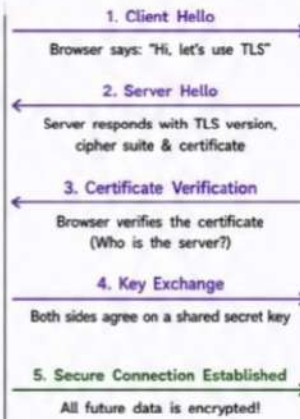


Look for the padlock in your browser!
It means the connection is secure.

THE TLS HANDSHAKE (HIGH LEVEL)



Your Browser



Google Server

WHAT IS A CERTIFICATE?

A digital ID card for the website.
It contains:

- ✓ Domain name
- ✓ Organization
- ✓ Public key
- ✓ Issued by a Trusted CA

Without a valid certificate, your browser will show a warning!

CERTIFICATE VERIFICATION FLOW



BROWSER CHECKS:

- ✓ Is the certificate issued by a trusted CA?
- ✓ Is the domain name correct?
- ✓ Is the certificate expired?
- ✓ Is the certificate revoked?

Only if all checks pass, the lock appears!

ENCRYPTION IN ACTION

Before Encryption (Readable)

```
GET /search?q=linux HTTP/1.1
Host: google.com
User-Agent: Chrome/124.0
Cookie: SID=abc123xyz
...
```



Anyone on the network can read this!



After Encryption (Encrypted)

```
x7f1k9$#da2@..
Q!zP8mKvR9...
kL91aa7*Qw2...
...
```

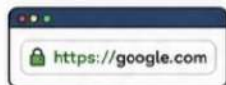


Only the server can decrypt and read it!



Encryption turns readable data into gibberish for everyone except the intended server.

HOW THE BROWSER SHOWS THE SECURE CONNECTION



You See
The Padlock



Certificate
Is Valid



Connection
Is Encrypted



Secure
HTTPS



You Are Safely Connected
To Google!

USEFUL COMMANDS TO CHECK HTTPS

Check Certificate Details

```
$ openssl s_client -connect google.com:443 -servername google.com
...
subject=CN = www.google.com
issuer=CN = GTS CA 1C3, O = Google Trust Services
SSL-Session:
  Protocol  : TLSv1.3
  Cipher    : TLS_AES_256_GCM_SHA384
  Verify return code: 0 (ok)
```

Quick Check with curl

```
$ curl -I https://google.com
HTTP/2 200
content-type: text/html; charset=ISO-8859-1
strict-transport-security: max-age=31536000
...
```

TLS VERSIONS (YOU SHOULD KNOW)

- TLS 1.0 / 1.1 Depreciated (Not Secure)
- TLS 1.2 Secure (Still widely used)
- TLS 1.3 Most Secure & Fastest

Modern browsers and servers prefer TLS 1.3 whenever possible.

WHY HTTPS MATTERS

- ✓ Protects passwords, cookies, and tokens
- ✓ Prevents eavesdropping on public Wi-Fi
- ✓ Prevents tampering (Man-in-the-Middle attacks)
- ✓ Builds trust with users and search engines

COMMON HTTPS ISSUES

- ⚠ Certificate expired
- ⚠ Self-signed certificate
- ⚠ Domain mismatch
- ⚠ Untrusted CA
- ⚠ TLS version too old
- ⚠ Clock on server is wrong



These issues break the trust and your browser shows warnings.



KEY TAKEAWAY

HTTPS uses TLS to encrypt, verify, and protect your communication. ✓



QUESTION FOR YOU

Have you ever seen a certificate warning in your browser? What was the reason? 🙋



NEXT STEP (5/5)

Now that the connection is secure, let's send an HTTP request and get a response from Google! 🚀

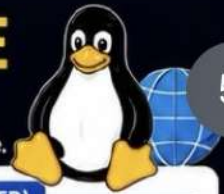


5/5

STEP 4: HTTP REQUEST & RESPONSE

"Let's Get the Data"

The connection is secure. Now your browser asks for data and gets a response.



5/5

1. HTTP REQUEST (FROM BROWSER)

Your browser sends an HTTP request to Google's server.



Your Browser

```
GET / HTTP/2
Host: www.google.com
User-Agent: Chrome/124.0.0.0
Accept: text/html,application/xhtml+xml
Accept-Language: en-US,en;q=0.9
...
```

(Asking for the homepage)



Google Server



KEY TAKEAWAY:

The request includes the path, headers, and other metadata about what the browser wants.

2. HTTP RESPONSE (FROM SERVER)

Google's server processes the request and sends back a response.



Google Server

```
HTTP/2 200 OK
Content-Type: text/html; charset=UTF-8
Date: Fri, 17 May 2024 10:20:30 GMT
Content-Length: 123456
...
```

<html> ... (HTML content) ... </html>



Your Browser



Status code 200 OK means success.

The body contains the HTML of the page.

WHAT HAPPENS INSIDE GOOGLE'S SERVER? (NGINX + BACKEND)



Request Arrives



Nginx (Web Server)



Application (Backend)



Database (Optional)



Generate Response



Send Response Back



NGINX'S ROLE

- ✓ Accepts the connection
- ✓ Handles TLS (HTTPS)
- ✓ Serves static files (CSS, JS, Images)
- ✓ Forwards requests to backend
- ✓ Returns the response



Nginx is like the traffic cop that sits in front of the application. It handles thousands of connections efficiently.

Example Nginx Config

```
server {
    listen 443 ssl;
    server_name www.google.com;

    location / {
        proxy_pass http://backend;
        proxy_set_header Host $host;
    }
}
```

Why Use Nginx?

- ⚡ High performance
- 👥 Handles many connections
- ⚖️ Load balancing
- 🛡️ Security (TLS, headers)
- 💾 Caching

THE COMPLETE JOURNEY (FROM URL TO WEBPAGE)

1. You Type URL
Enter google.com in your browser2. DNS Resolution
Browser finds Google's IP address3. TCP Handshake
A reliable connection is established4. TLS Handshake
A secure, encrypted connection is created5. HTTP Request
Browser requests the webpage6. Server Processing
Nginx and backend process the request7. HTTP Response
Server sends back HTML, CSS, JS, etc.8. Page Loads
Browser renders the webpage

FUN FACT:

A single page load may trigger dozens of additional requests (CSS, JS, fonts, images, ads, analytics, etc.) Each may repeat parts of this journey!

USEFUL COMMANDS (LINUX)

```
dig google.com    DNS lookup

curl -I https://google.com    Check HTTP headers

curl -v https://google.com    Verbose request/response

openssl s_client -connect google.com:443 -servername google.com

Test TLS handshake

ss -tnp | grep :443

Check active connections
```

TOOLS TO VISUALIZE

```
ping google.com    Check connectivity

tracert google.com    See the network path

curl -w "%{time_total}s\n" https://google.com    Check response time

chrome://net-internals/#events    Detailed network events
```

COMMON HTTP STATUS CODES

200	OK	Request successful
301	Moved Permanently	Redirect
302	Found	Temporary redirect
400	Bad Request	Invalid request
401	Unauthorized	Authentication required
403	Forbidden	Access denied
404	Not Found	Resource not found
500	Internal Server Error	Server-side error
503	Service Unavailable	Server is temporarily down

KEY TAKEAWAYS

- ✓ Typing google.com triggers multiple protocols working together.
- ✓ DNS translates domain to IP.
- ✓ TCP creates a reliable connection.
- ✓ TLS encrypts the communication.
- ✓ HTTP is used to request and receive data.
- ✓ Nginx acts as a high-performance reverse proxy.
- ✓ Many tiny pieces work together to make one webpage appear.



SAVE THIS POST

You'll want to refer back to this journey!



QUESTION FOR YOU

Which part of this journey did you find most interesting? Comment below! 🗣️

BONUS PREVIEW

Next Series: How Google Finds the Fastest Server Near You! 🌐

